



AI-Powered Data Engineering with Lakeflow

Frank Munz, Databricks

+ Databricks SMEs
September 2026

©2022 Databricks Inc. — All rights reserved



Housekeeping



Ask questions in the Q&A



Engage in the polls



**Recording
shared post event**



Lab access instructions

Access the lab environment:

- Use sign up link: <https://go.vocareum.com/SYU-NYD-QXS>
- Code (if asked): [de-workshop-sept](#)
- Post-event email with recording, slides and an updated access link will be shared within 48 hours

Best practices:

- Incognito is not supported
- We recommend using Chrome or Firefox. DO NOT USE Safari has a known issue with the pop-ups being blocked and crash behavior
- Enable pop-ups from Databricks Academy (this is a Docebo/LMS requirement)
- Enable cookies (Browser Support and Requirements | Vocareum – Documentation)
- DO NOT end the lab. You will receive a code after the workshop to engage in the lab again with the slides & recording.



Welcome & Signup for Lab Environment

- GitHub repo used later in labs:
<https://github.com/databricks/tmm>
& folder: [Lakeflow-DataEng-Workshop](#)
- Lab Guide will be in your workshop environment. Online, you can find it here:
<https://github.com/databricks/tmm/blob/main/Lakeflow-DataEng-Workshop/Labguide.md>



AI-Powered Data Engineering with Lakeflow

Frank Munz, Databricks

+ Databricks SMEs

DAIS 2026

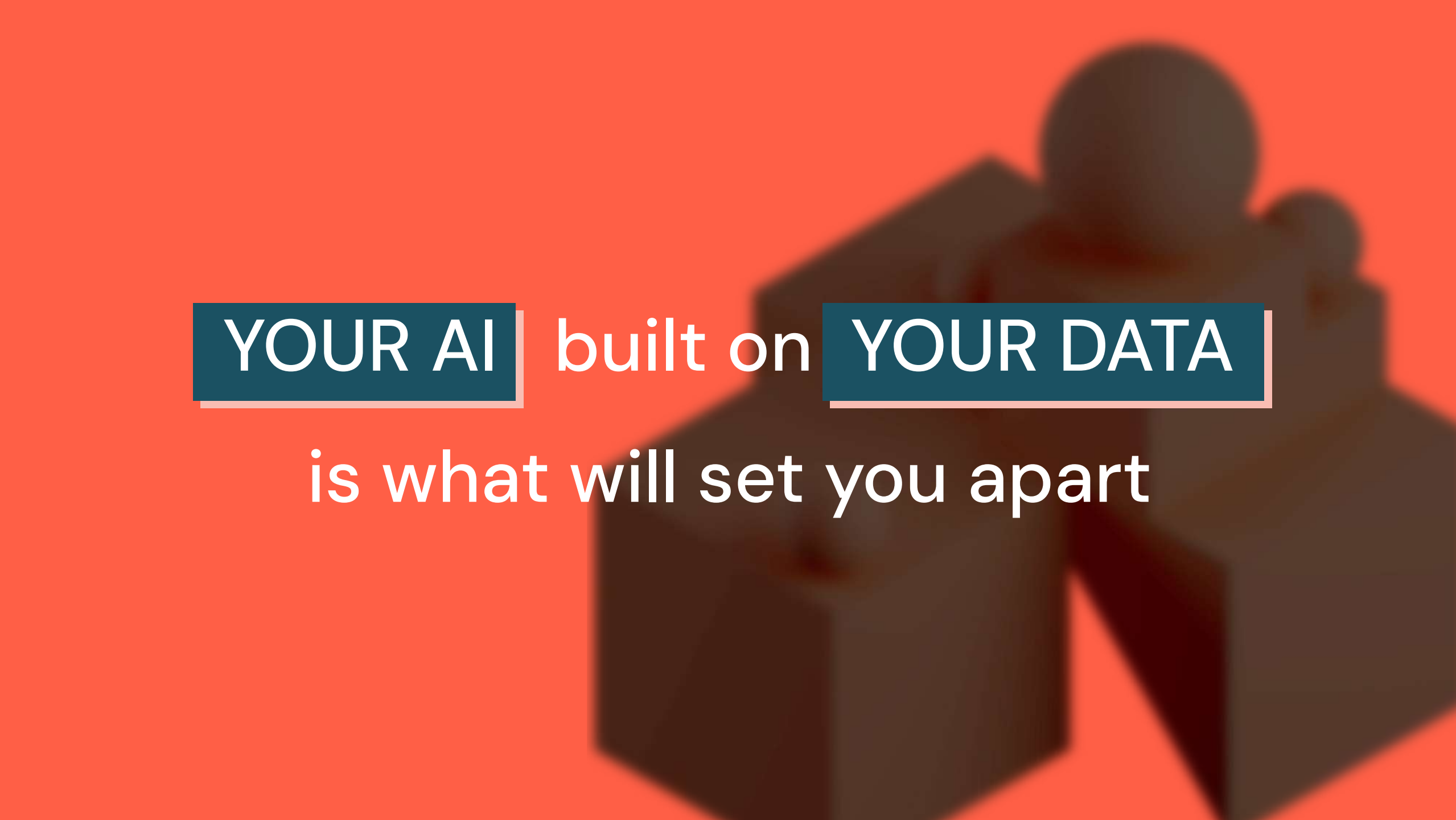
©2022 Databricks Inc. — All rights reserved



About Frank

- Principal TM Engineer, 5y @ Databricks
- Based in Munich,    
- All things large scale data & compute
- Formerly AWS Tech Evangelist, SW architect, data scientist





YOUR AI built on **YOUR DATA**

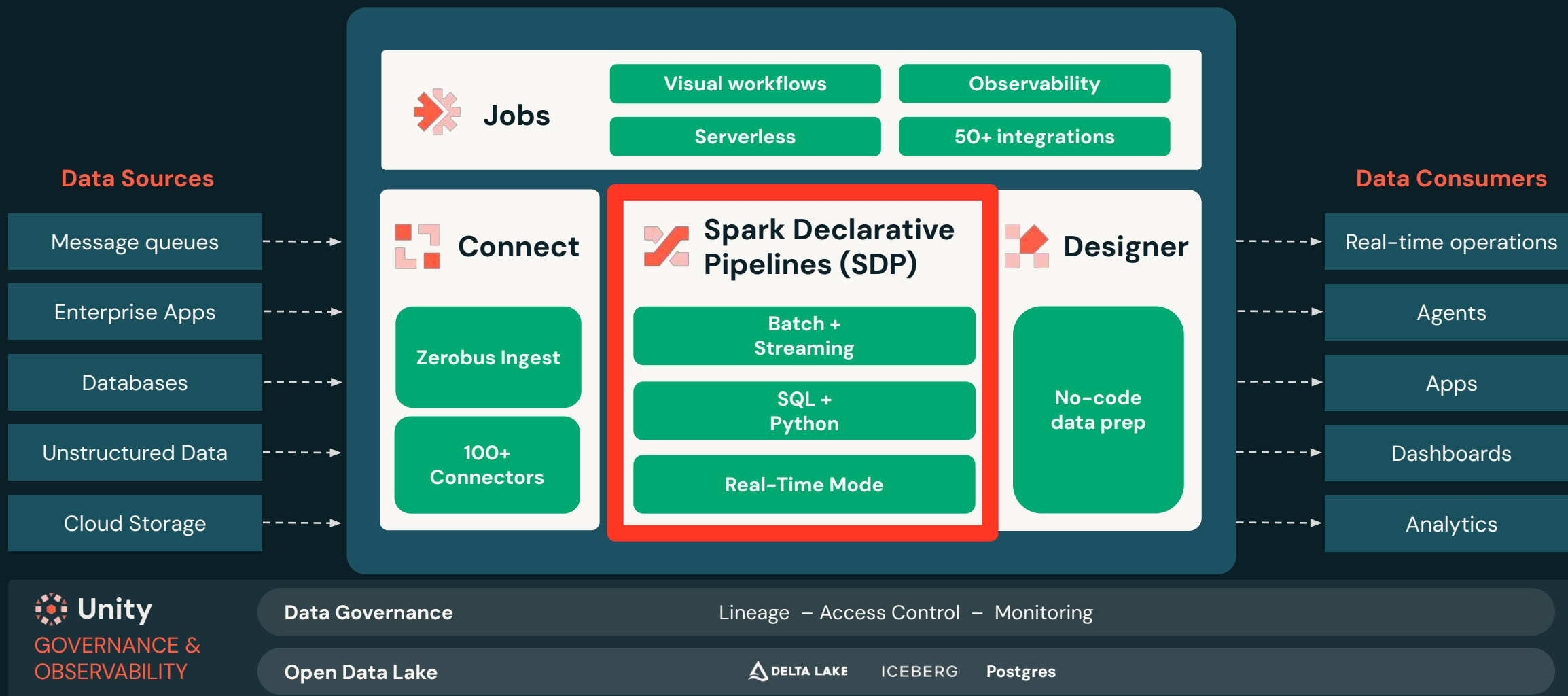
is what will set you apart

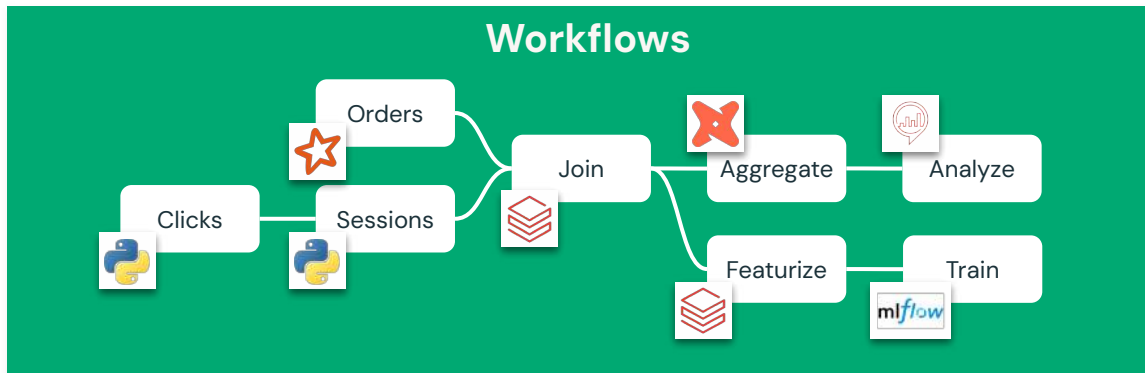
The background features several large, solid blue geometric shapes: a circle in the top right, a square in the top right corner, a triangle in the middle right, a square in the bottom right, and a triangle in the bottom right corner. These shapes are arranged in a way that they partially overlap and frame the central text.

Agents will change the world,
but they require a platform



Lakeflow: Powering the Agent-Ready Data Platform





BI & Data
Warehousing



Data
Engineering



Data
Streaming



Data
Science & ML

Unity Catalog

Delta Lake



Lakeflow Jobs

Unified orchestration for data, analytics, and AI on the Lakehouse Platform

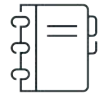
- Simple authoring
- Actionable insights
- Proven reliability

YipitData: Why we migrated from
[Airflow to Workflows](#)

Building Blocks of Lakeflow Jobs

A unit of orchestration in Databricks Workflows is called a **Job**.

Jobs consist of one or more **Tasks**



Databricks Notebooks



Python Scripts



Python Wheels



SQL Files/Queries



DBSQL Dashboards



Spark Declarative Pipelines



dbt



Java JAR file



Spark Submit

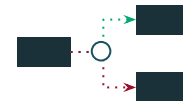
Control flows can be established between **Tasks**.



Sequential



Parallel



Conditionals (Run If)



Jobs-as-a-Task (Modular)



For-Each Loop

Jobs supports different **Triggers**



Manual Trigger



Scheduled (Cron)



API Trigger



File Arrival Triggers



Table Triggers



Continuous (Streaming)

Pipelines Agents Can Build

Open



Unify batch and streaming on Apache Spark to eliminate vendor lock-in and leverage broad LLM training context

Safe



Catch errors upfront with compiler dry runs, dependency inference, and inline data quality expectations

Governed



Enforce enterprise boundaries with Unity Catalog governance, Genie Ontology context, and smart execution

Programming with Declarative Pipelines

Say what should be done, not how to do it

Spark Imperative Program

```
def upsertToDelta(microBatchOutputDF: DataFrame, batchId:
Long) {
  microBatchOutputDF
    .groupBy("key")
    .agg(max_by("ts", struct("*").alias("row")))
    .select("row.*")
    .createOrReplaceTempView("updates")

  microBatchOutputDF.sparkSession.sql(s"""
    MERGE INTO cdc_data_raw t
    USING updates s
    ON s.key = t.key
    WHEN MATCHED AND s.is_delete THEN
      UPDATE SET DELETED_AT=now()
    WHEN MATCHED THEN UPDATE SET
      A=CASE WHEN s.ts > t.ts THEN s.a ELSE t.a,
      B=CASE WHEN s.ts > t.ts THEN s.b ELSE t.b,
      ... for every column ...
    WHEN NOT MATCHED THEN INSERT *
  """)
}

cdcData.writeStream
  .foreachBatch(upsertToDelta _)
  .outputMode("append")
  .start()
```

Declarative Pipelines Program

```
CREATE OR REFRESH STREAMING TABLE cdc_data;

CREATE FLOW apply_cdc AS AUTO CDC INTO
cdc_data
FROM source_data
KEYS (id)
APPLY AS DELETE WHEN is_deleted
SEQUENCE BY ts
```



Lakeflow Pipeline Editor

The screenshot displays the Lakeflow Pipeline Editor interface. On the left is a 'Workspace' panel with a tree view showing folders like 'data_sources', 'explorations', 'transformations', and 'utilities'. The main area is divided into three sections: a code editor on the left showing a Python script for a DLT pipeline, a DAG (Directed Acyclic Graph) in the center showing the pipeline's flow, and a data preview table at the bottom.

Code Editor: The script defines a pipeline named 'sample_taxi_trips' with two tables: 'sample_taxi_trip_raw' and 'sample_taxi_trip_silver'. It uses DLT (Data Lake Tool) for reading and writing data.

```
1 import dlt
2 from pyspark.sql.functions import *
3
4 import sys, os
5 sys.path.append(os.path.abspath('../utilities'))
6 from utils import distance_km
7
8 @dlt.table(name="sample_taxi_trip_raw",
9           comment="Table with raw data on NYC taxi trips")
10 def sample_taxi_trip_raw():
11     return dlt.readStream("sample_taxi_source")
12
13 @dlt.table(name="sample_taxi_trip_silver",
14           comment="Taxi trip data with transformed columns")
15 @dlt.expect_or_drop("valid_distance", "trip_distance_miles > 0.0")
16 def sample_taxi_trip_silver():
17     return (
18         dlt.read("sample_taxi_trip_raw")
19         .withColumnRenamed("tpep_pickup_datetime", "pickup_datetime")
20         .withColumnRenamed("tpep_dropoff_datetime", "dropoff_datetime")
21         .withColumnRenamed("total_amount", "total_amount_usd")
22         .withColumnRenamed("trip_distance", "trip_distance_miles")
23         .withColumn("trip_distance_km", distance_km(col
```

DAG: The DAG shows the pipeline's flow from the 'sample_taxi_trip_raw' table to the 'sample_taxi_trip_silver' table, and then to the 'sample_taxi_trip_gold' table.

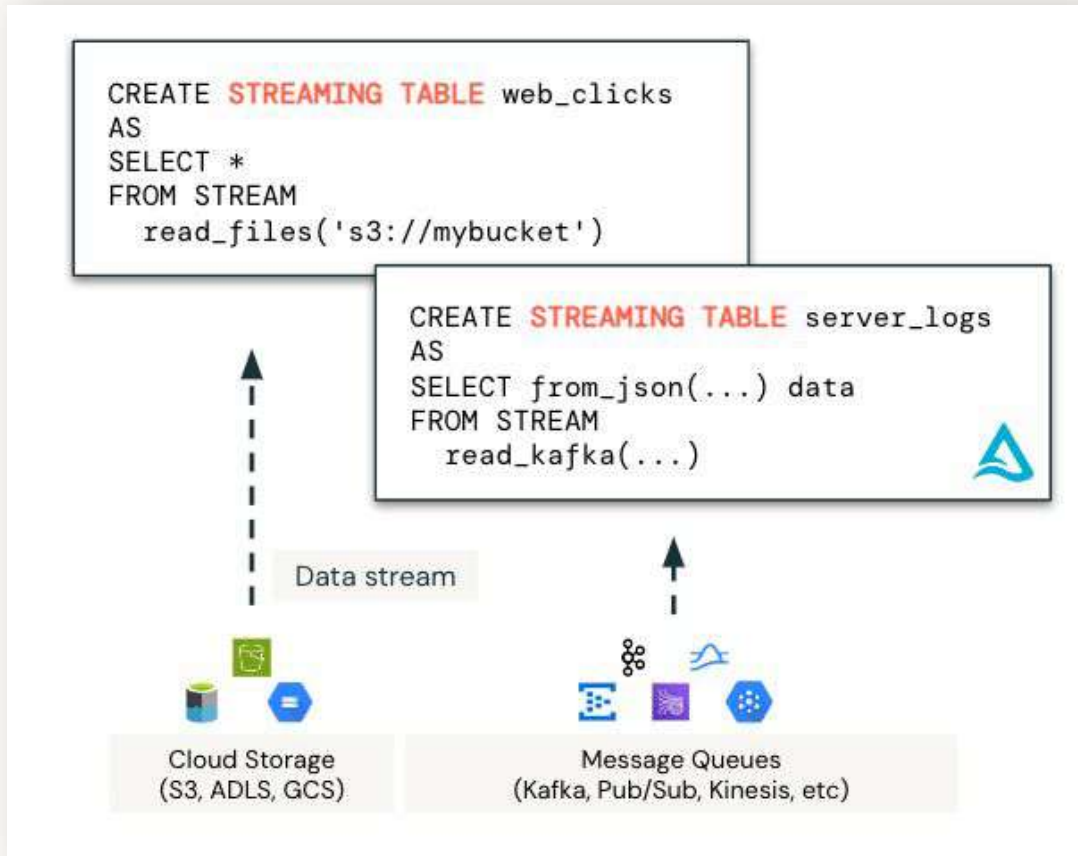
Data Preview: The table shows a preview of the data, with columns including 'DOLocationID', 'PULocationID', 'RatecodeID', 'VendorID', '1.2 congestion_surcharge', '1.2 extra', '1.2 fare_amount', '1.2 improvement_surcharge', and '1.2 mta_fare'.

1. Pipeline asset browser
 - Default folder structure
 - Automatic Pipeline config updates
2. Multi-file code editor with features for step-by-step development
 - Soft tabs
 - Gutter actions
 - Inline error indicators
3. Pipeline-specific toolbars
4. Interactive DAG
5. Data previews
6. Execution insights panels

Streaming Table (ST)



A **simple way** to ingest
and process **streaming**
data



→ Exactly once data ingestion, append only sources

Materialized View (MV)



```
CREATE MATERIALIZED VIEW customer_orders
AS
SELECT
  customers.name,
  sum(orders.amount),
  orders.orderdate
FROM orders
LEFT JOIN customers ON
  orders.custkey = customers.c_custkey
GROUP BY
  name,
  orderdate;
```



Results are
pre-computed and
incrementally
refreshed

customers
(Table)



orders
(Table)



**Incrementally refreshed
transformations for ETL and
dashboards**

→ Fast Dashboards, complex JOINS / AGGs, downstream usage

Flows

Define How Data is Processed

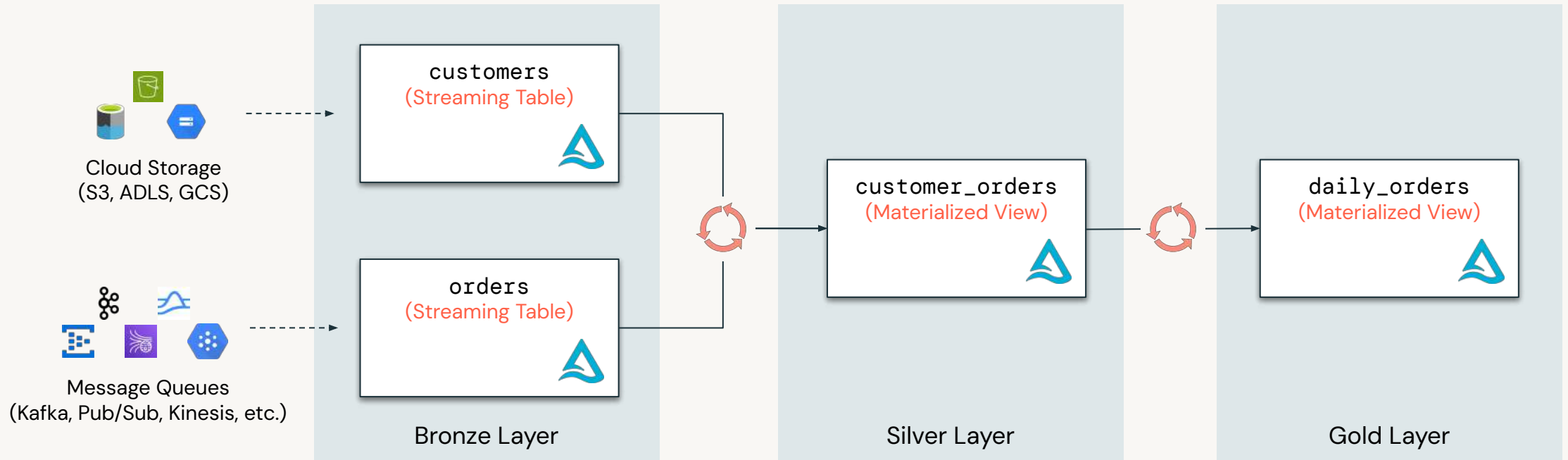
- A flow moves data to a table
- Supports batch and streaming
- STs and MVs define an implicit default flow automatically
- Explicit flows enable fan-in, AUTO CDC, and REPLACE WHERE into one streaming table

```
-- Fan-in: many append flows → one streaming table
CREATE STREAMING TABLE kafka_target;
CREATE FLOW t1 AS INSERT INTO kafka_target BY NAME
    SELECT * FROM read_kafka(subscribe ⇒ 'topic1');
CREATE FLOW t2 AS INSERT INTO kafka_target BY NAME
    SELECT * FROM read_kafka(subscribe ⇒ 'topic2');

-- AUTO CDC bitemporal: business + system time (CDC-only target)
CREATE STREAMING TABLE customers;
CREATE FLOW c AS AUTO CDC INTO customers
    FROM STREAM(cdc_raw) KEYS (id)
    SEQUENCE BY business_ts SYSTEM SEQUENCE BY system_ts
    STORED AS BITEMPORAL;

-- REPLACE WHERE: overwrite just a predicate slice (Databricks SQL)
CREATE OR REFRESH STREAMING TABLE orders_enriched
    FLOW REPLACE WHERE date ≥ '2026-07-01' BY NAME
    SELECT * FROM orders o JOIN products p ON o.pid = p.pid;
```

Linking STs and MVs to build data pipelines: Medallion Architecture



Streaming Data Pipeline

SDP ST/MV Python Example

Easily ingest records from **message buses**

```
from pyspark import pipelines as dp

#streaming table
@dp.table
def kafka_data():
    return spark.readStream
        .option( "format" , "kafka" )
        .option( "subscribe" , "events" )
        .load()

#materialized view
@dp.materialized_view
def events_by_topic():
    return spark.read.table("kafka_data")
```

...

This example creates a ST and MV with all the records published to the Kafka topic “events”.

- The Kafka source + SDP **automatically track which partitions / offsets** have already been read.
- **Any structured streaming source included in DBR** can be used with pipelines
- Message buses provide **the lowest latency** for ingesting data
- SDP materialized view aggregates the data

Change Data Capture (CDC) from RDBMS

A variety of 3rd party tools can provide a streaming change feed



Serverless Compute



SIMPLE and FAST

No knobs
Fast startup
For any practitioner



EFFICIENT

Fully managed and versionless
Paying only for what you use
Strong cost governance



RELIABLE

Secure by default
Smart fail-overs

DB SQL

Data
Engineering

AI

Notebooks

Serverless Compute

Hands-off auto optimized
compute managed by Databricks

Storage

SDP with Real-time Mode RTM

RTM settings for SDP

- single/double digit ms latency
- same Spark platform
→ makes external systems like Flink redundant
- Use source (message bus, rate), sinks (message bus, Lakebase, console)
- On the flow, for transformation use `@dp.update_flow(...)`

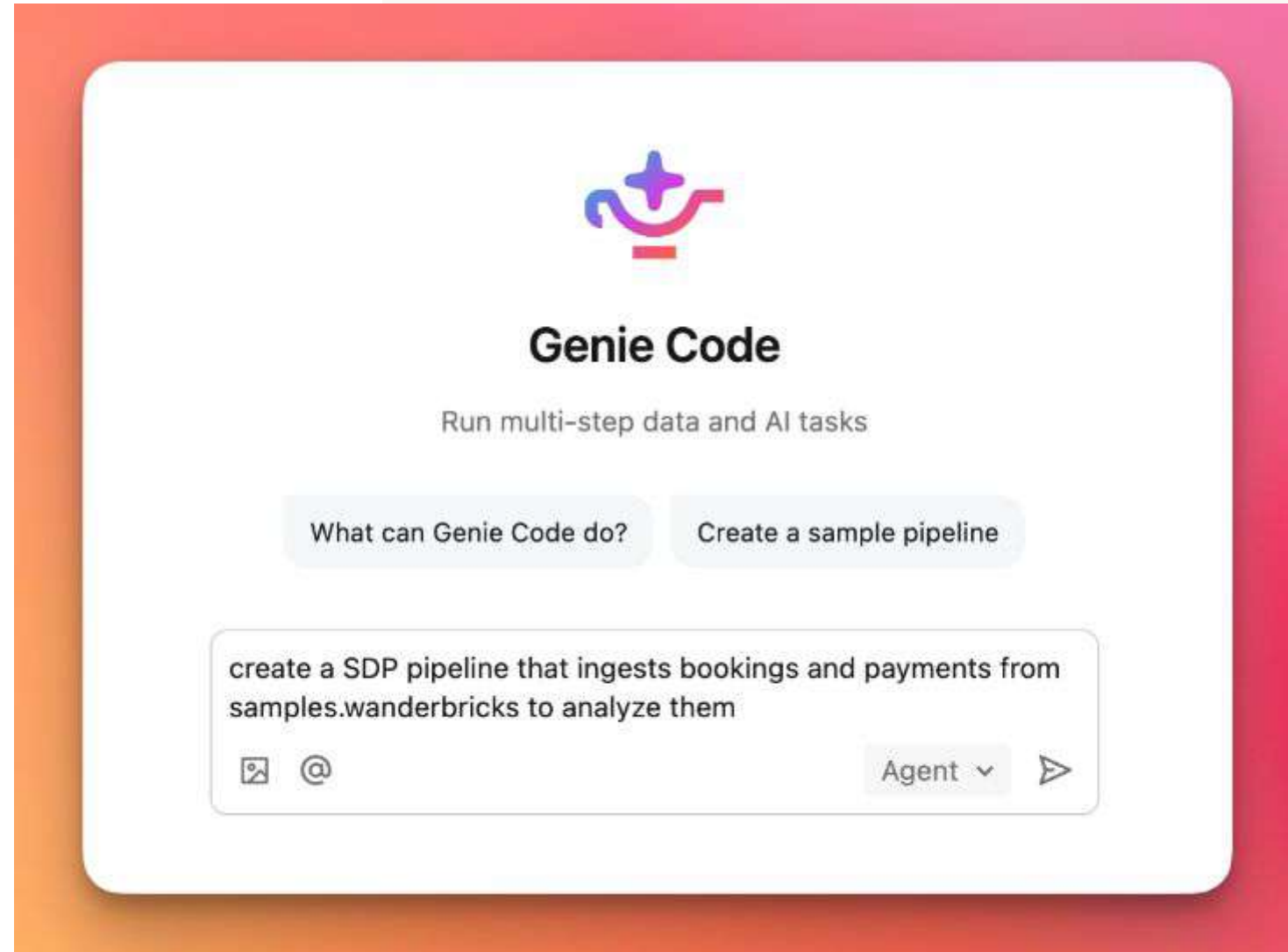
```
dp.create_sink(  
    "hot_temperatures_sink",  
    "console",  
    {"mode": "append", "truncate": "false"},  
)  
  
@dp.update_flow(  
    name="temperature_rtm_flow",  
    target="hot_temperatures_sink",  
    spark_conf={  
        "pipelines.trigger": "RealTime",  
        "pipelines.trigger.interval": "2 minutes",  
    },  
)  
def temperature_rtm_flow():  
    return (  
        spark.readStream  
        .format("rate")  
        .option("rowsPerSecond", "100")  
        .load()  
        .withColumnRenamed("timestamp", "source_timestamp")  
    )
```



Genie Code

An autonomous AI agent
for Data Science,
Engineering and Analytics

- Natural language prompt
- Creates, manages and debugs full SDP and Lakeflow Jobs
- [Agent skills for Claude Code are public](#)
- Can be extended with Skills and MCP servers



Data Integration: Lakehouse Federation

Simply integrate with Oracle, Snowflake, Glue, MySQL, BigQuery, Postgres, ...

The screenshot displays the Databricks Catalog Explorer interface. On the left, the 'Catalog' sidebar shows a tree view of data sources under 'dbdemo Serverless'. The 'bigquery' source is expanded, showing various datasets, with 'stocks' selected at the bottom. The main panel, titled 'Catalog Explorer > bigquery > uswest >', shows the 'stocks' view. It includes a search bar, a filter icon, and buttons for 'Open in a dashboard', 'Share', and 'Create'. Below these are tabs for 'Overview', 'Sample Data', 'Details', 'Permissions', 'History', 'Lineage', and 'In...'. The 'Sample Data' tab is active, displaying a table of Google stock data. Above the table is a banner for 'Ask Genie' with the text 'Explore data in natural language with Genie AI'. The table has columns for 'Ticker', 'Date', '1.2 Open', '1.2 High', and '1.2 Low', with 12 rows of data.

	Ticker	Date	1.2 Open	1.2 High	1.2 Low
1	GOOG	2004-08-19	50.0	52.03	47.98
2	GOOG	2004-10-15	72.0	72.75	70.97
3	GOOG	2004-11-10	84.0	86.26	83.16
4	GOOG	2004-11-01	96.0	98.83	95.63
5	GOOG	2005-10-19	152.0	154.93	151.97
6	GOOG	2010-09-15	240.0	240.94	239.25
7	GOOG	2008-06-25	272.0	278.89	271.83
8	GOOG	2008-04-28	272.0	278.4	269.5
9	GOOG	2011-03-29	288.0	290.94	286.5
10	GOOG	2011-11-14	304.0	309.04	303.89
11	GOOG	2012-08-03	320.0	321.85	318.07
12	GOOG	2007-10-26	336.0	338.26	334.03

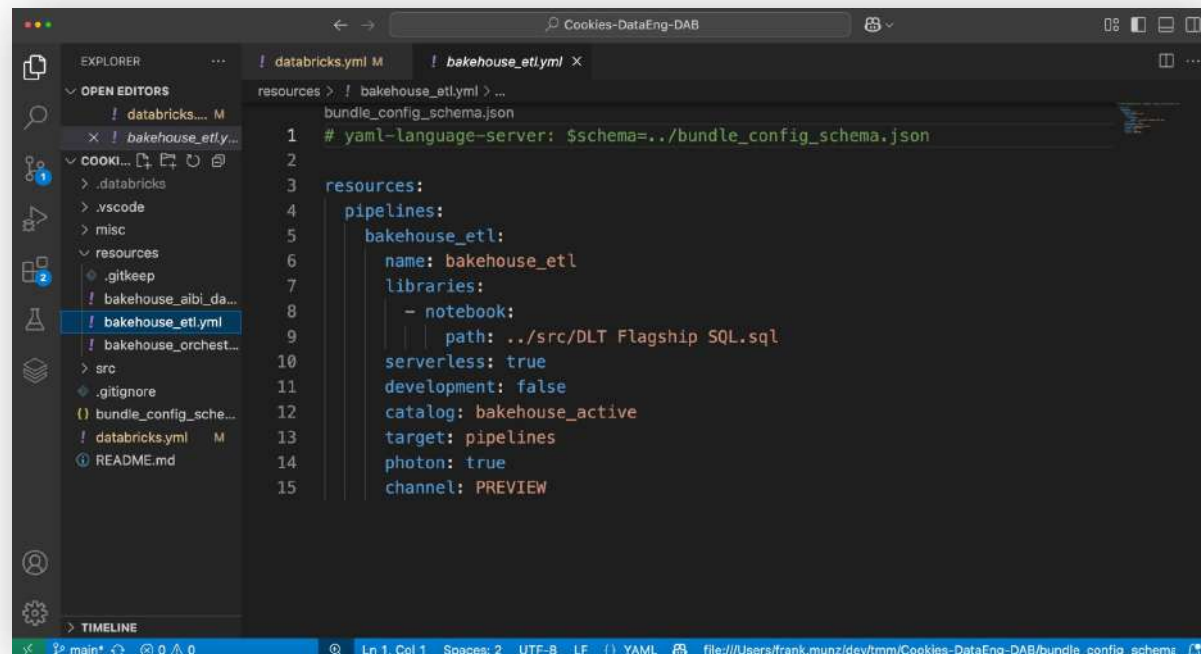


VSCode and Declarative Automation Bundles (DAB)

CI/CD

- YAML-based
- Specify and deploy notebooks, pipelines, and jobs across environments
- VSCode support with plugin
- Databricks CLI or Git actions

```
$ databricks bundle validate  
$ databricks bundle deploy -t prod
```



Alternative
options



Python SDK

The Jobs Python API allows you to create, edit, and delete jobs



Terraform Provider

The databricks_jobs resource can be used to create and manage Jobs with Terraform

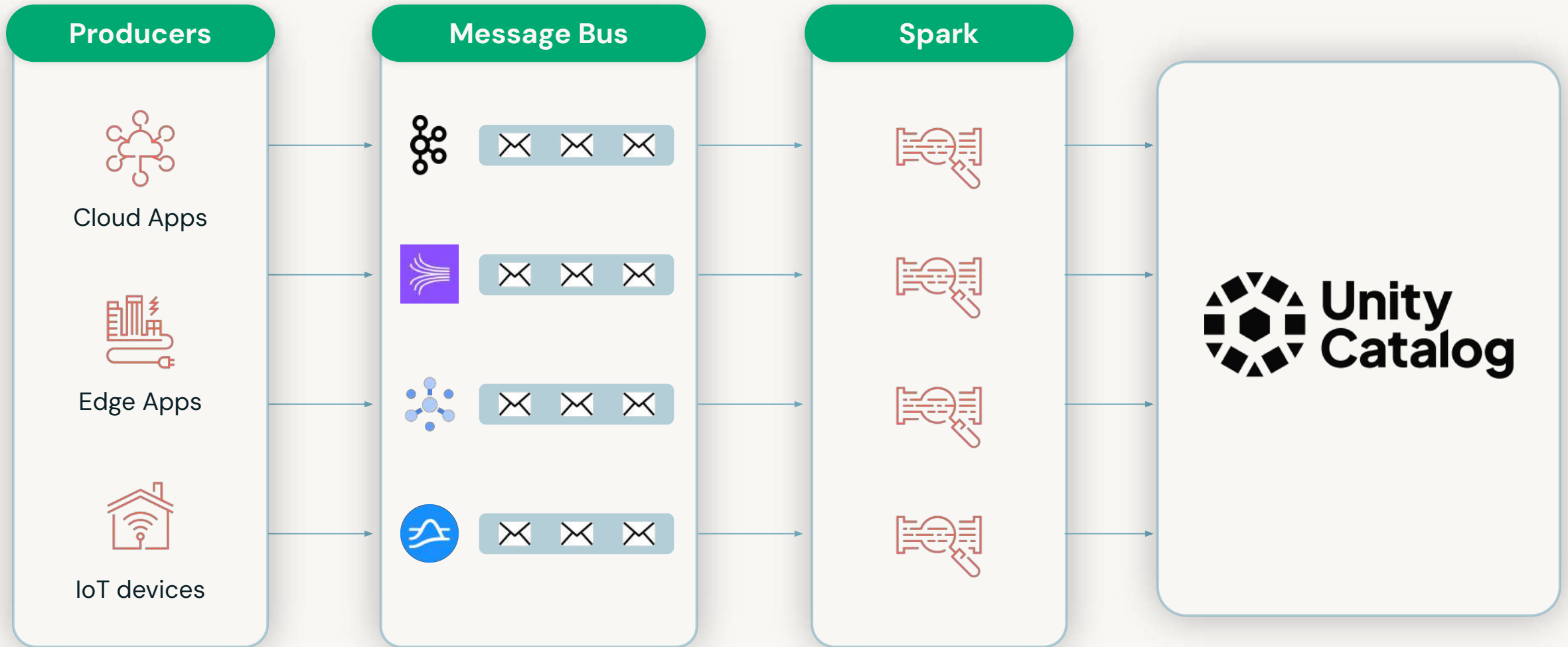


Databricks Jobs REST API

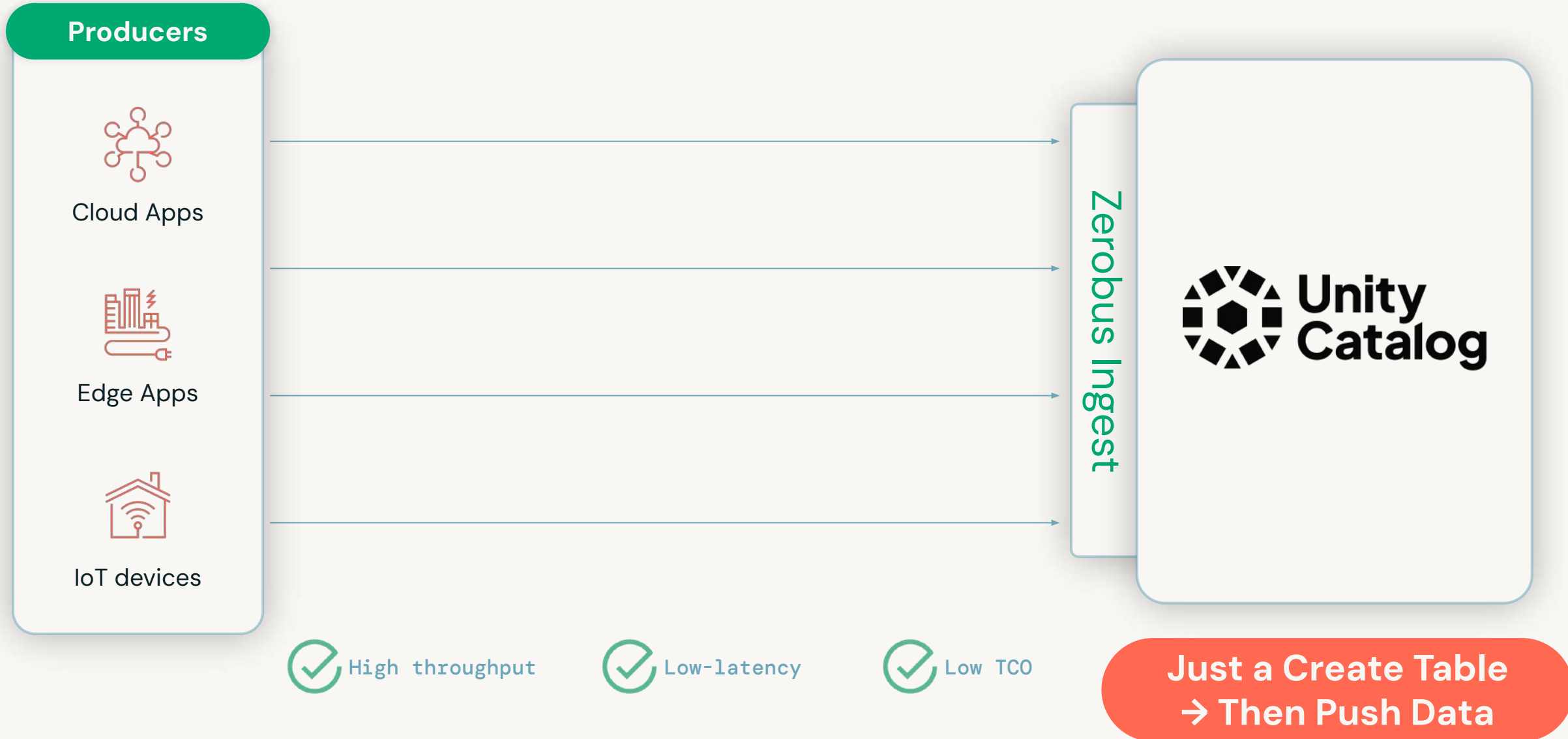
Low-level REST API allows you to manage Jobs programmatically



Message Bus



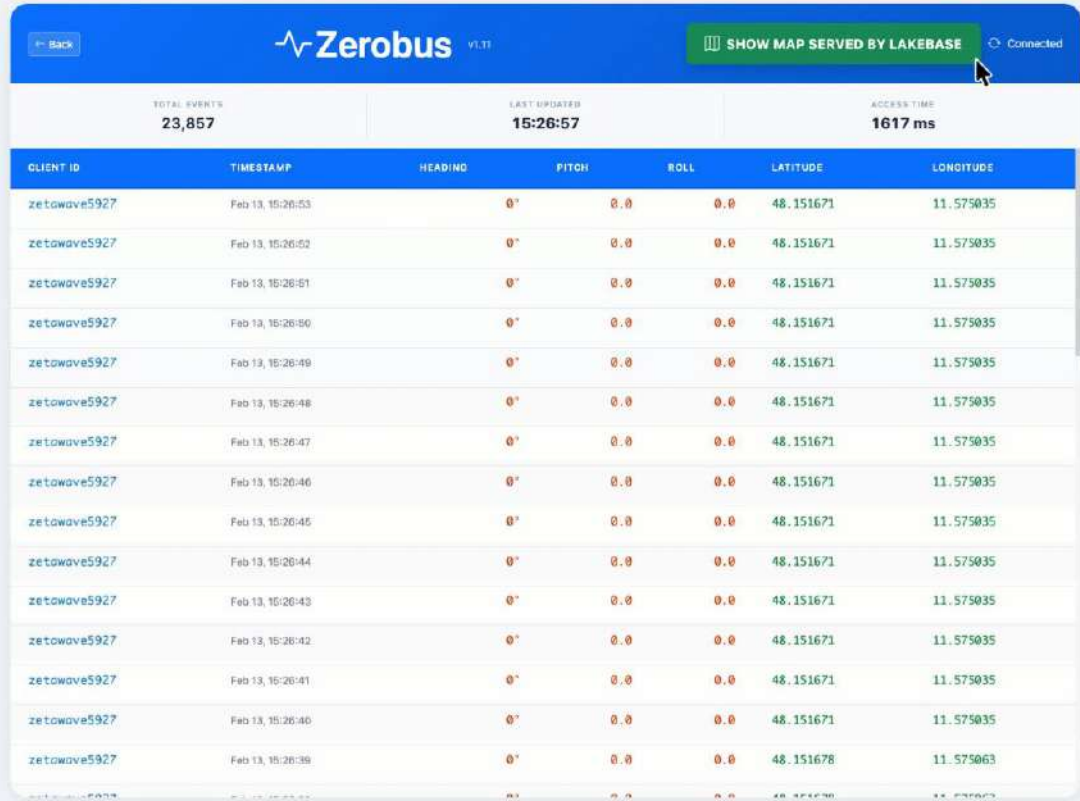
Zerobus Ingest



Zerobus Ingest

Push event data from clients to tables without Kafka

- **writes** < 5 sec latency
- throughput > 100 MB/sec per cx
- **highly concurrent** workloads.
- without cost or complexity of a message bus.
- governed by Unity Catalog
- powered by serverless compute
- gRPC, REST + Python, Java, Rust SDK



The screenshot shows the Zerobus dashboard interface. At the top, there's a blue header with the Zerobus logo and version v1.11. A green button labeled 'SHOW MAP SERVED BY LAKEBASE' and a 'Connected' status indicator are on the right. Below the header, three summary metrics are displayed: 'TOTAL EVENTS 23,857', 'LAST UPDATED 15:26:57', and 'ACCESS TIME 1617 ms'. The main part of the dashboard is a table with columns: CLIENT ID, TIMESTAMP, HEADING, PITCH, ROLL, LATITUDE, and LONGITUDE. The table contains 15 rows of data for client 'zetawave5927', showing timestamps from Feb 13, 15:26:53 to Feb 13, 15:26:39. The heading, pitch, and roll values are consistently 0°, 0.0, and 0.0 respectively. The latitude is 48.151671 for most rows, and the longitude is 11.575035 for most rows, with a slight variation in the last row.

CLIENT ID	TIMESTAMP	HEADING	PITCH	ROLL	LATITUDE	LONGITUDE
zetawave5927	Feb 13, 15:26:53	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:52	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:51	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:50	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:49	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:48	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:47	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:46	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:45	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:44	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:43	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:42	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:41	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:40	0°	0.0	0.0	48.151671	11.575035
zetawave5927	Feb 13, 15:26:39	0°	0.0	0.0	48.151678	11.575063



ANNOUNCING



Genie

ZERO  OPS

Put your data and AI ops on autopilot

Genie ZeroOps

Less time maintaining – more time building



Detect

Per-table models,
continuously fine-tuned

Metrics, events, logs



Assess

Graph ranking using
Unity Catalog lineage

Supervisor and research
sub-agents



Remediate

Code generation using
Genie Code

Unity Catalog policy
recommendations



Verify

Shallow clones, similar
to Lakebase branches

Code and network
isolation



Lab access instructions

Access the lab environment:

- Use sign up link: <https://go.vocareum.com/HAU-UKQ-HNN>
- Code (if asked): **de-workshop-sept**
- Post-event email with recording, slides and an updated access link will be shared within 48 hours

Best practices:

- Incognito is not supported
- We recommend using Chrome or Firefox. DO NOT USE Safari has a known issue with the pop-ups being blocked and crash behavior
- Enable pop-ups from Databricks Academy (this is a Docebo/LMS requirement)
- Enable cookies (Browser Support and Requirements | Vocareum – Documentation)
- DO NOT end the lab. You will receive a code after the workshop to engage in the lab again with the slides & recording.



Lakeflow Demos You Should Try!



Getting Started with Spark Declarative Pipelines: OSS + Lakeflow

How to get started with Spark Declarative Pipelines (SDP)



Introduction

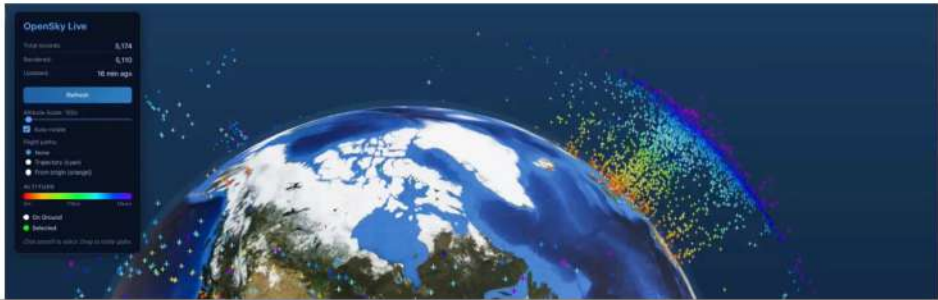
What is Spark Declarative Pipelines?

Spark Declarative Pipelines (SDP) tutorials

FAQ

Declarative [data pipelines](#) have become a native Spark capability in Apache Spark™ [4.1](#). No additional frameworks. No external dependencies. No new learning curve. Millions of existing Spark users can now build production-grade [ETL](#) pipelines, using the tools they already know and love.

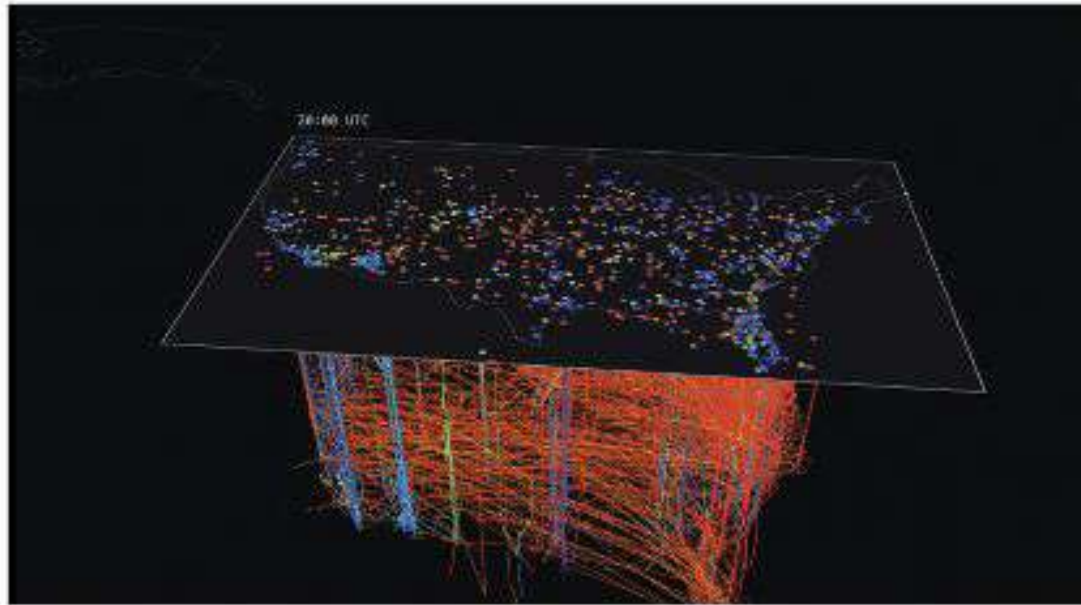
The example used here, a production-grade pipeline that tracks every aircraft in the sky with millions of live IoT events streaming in every second, used to be a serious engineering effort. Now it can be done in a coffee break, with a few lines of code and 100% open source.



[OSS and Lakeflow Tutorials \(realtime flight data\)](#)

Create a [Databricks Free Edition account](#), and add the avionics SDP demo to your portfolio on LinkedIn

Genie Tutorial for Data Engineers

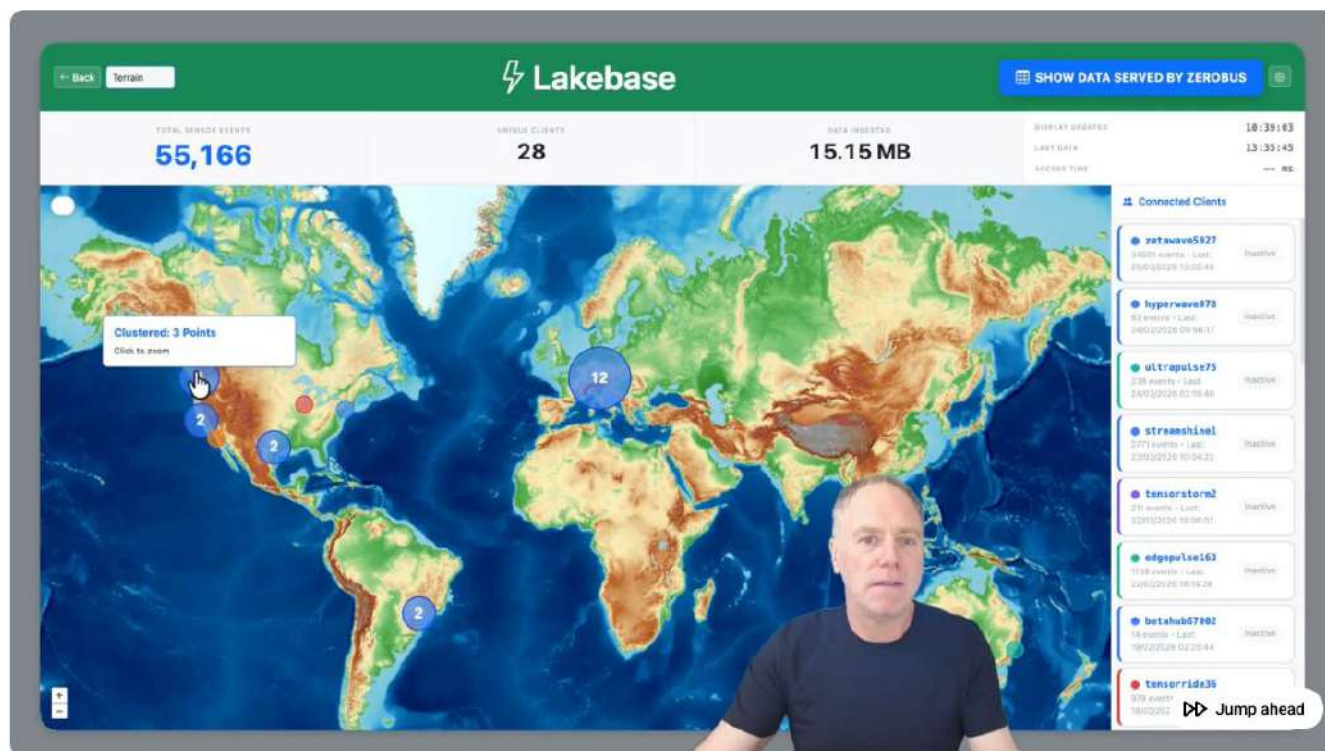


Tutorial: Databricks Genie for Data Engineers and Data Scientists

Tutorial:

Use Genie Agents and Genie Code with SDP and Job to process 700,000,000 records on Databricks Free Edition.

Zerobus Ingest Demo



Zerobus Ingest demo:
ingest streaming
IoT events without
Kafka from all over
the world

Convert Messy Sales Data to AI insights

databricks.com/demos

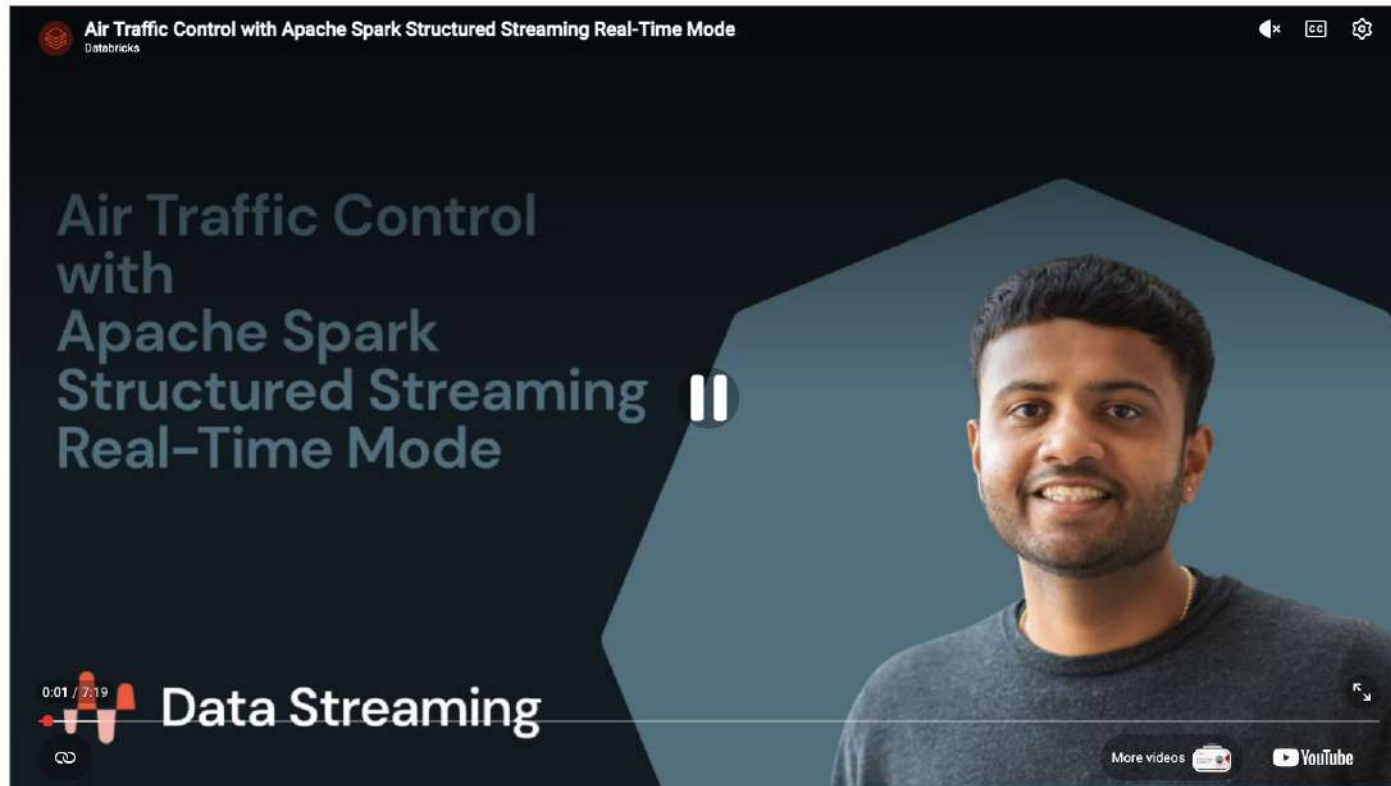


Use Lakeflow Connect, Jobs and SDP to create a marketing solution with AI for a global food corporation.

[Video Walkthrough](#) with [GitHub Repo](#) + DABs

Spark Streaming Real-time Mode (RTM)

Air Traffic Control With Apache Spark Structured Streaming Real-Time Mode



[Link](#)

Technical Questions?

Sign-up for the Community

Ask technical follow-up questions here:

<https://community.databricks.com/>

The screenshot shows the Databricks Community website. At the top is the Databricks Community logo and a navigation bar with links: Get Started With Databricks, Databricks Platform, Learning, Groups, Events, and Community Cove. Below the navigation bar is a welcome message: "Welcome to the Databricks Community. Learn, network, and celebrate with data practitioners from around the world." There is a search bar and a statistics bar showing 88356 Members, 1601 Online, and 38448 Posts. To the right of the welcome message is a graphic with several hexagonal frames containing photos of community members. Below this are three featured posts:

- Build knowledge with instructor-led training and save 20% today with code: ilt20**
524 views, 0 comments, 4 likes
on a week ago
- Databricks Onboarding Training (DOT) — Getting Started with Databricks**
2722 views, 3 comments, 9 likes
on 08-03-2023
- Attn: customers, join Databricks Champions. Stand out as a Data + AI leader.**
Let's share your story with the world, recognize your work, and grow your personal brand.
Join the Databricks Champions program
3140 views, 4 comments, 8 likes
on 07-19-2023



Try
Databricks
free

Thank You!

Air Traffic Control System

Spark Streaming, Real-time mode and Databricks Apps (REC)

